

TruthSeeker: AI-Powered Fact-Checking with Corrective RAG

Semester Project Report

Course: Introduction to Artificial Intelligence

Submitted to: Dr. Muhammad Ismail Mangrio — Assistant Professor & Coordinator (CS)

Institution: Sukkur IBA University

Submitted by:

Hassan Ali (023-23-0200)

Muhammad Haad (023-23-0199)

Date: June 2026

Table of Contents

Table of Contents

1. Task and Dataset Used

2. Data Pre-Processing Steps

3. Architecture of the Agent

4. Training Objective, Model Selection & Design Choices

5. Performance Evaluation

6. Application of Techniques Taught in Class

7. Main Challenges Faced and How They Were Addressed

8. References

1. Task and Dataset Used

1.1 Problem Statement and Context

In the modern digital age, the rapid dissemination of information across social networks and digital platforms has brought an unprecedented challenge: the widespread propagation of misinformation. Misinformation spreads faster than factual corrections, often leading to significant social, political, and personal consequences. Traditional large language models (LLMs) such as GPT-3 or Llama face a fundamental flaw when acting as knowledge repositories: they are prone to *hallucinations*, generating text that is highly plausible but factually incorrect.

Standard Retrieval-Augmented Generation (RAG) pipelines were introduced to mitigate this issue by grounding the generative model in an external database of verified information. However, standard RAG operates on an implicit assumption of trust: it assumes that whatever the retriever returns is relevant and correct. If the retrieval step fails to fetch pertinent information, the generator is forced to formulate a response based on irrelevant data, thus recreating the hallucination problem in a different form.

TruthSeeker addresses this fundamental gap by implementing a **Corrective RAG (CRAG)** pipeline. TruthSeeker acts as an intelligent fact-checking agent that not only retrieves evidence but also *self-evaluates* the quality and relevance of that evidence. When the internal knowledge base falls short, TruthSeeker automatically transitions to searching the live web, pulling from trusted fact-checking platforms, and ultimately rendering a structured, evidence-backed verdict.

1.2 Task Definition

The core objective of TruthSeeker is to verify the truthfulness of natural language claims provided by users. Given a claim (e.g., "*NASA confirmed the discovery of water on Mars in 2018*"), the system must autonomously execute the following operations:

1. **Information Retrieval:** Search and extract relevant evidence from a large-scale knowledge base of verified news articles.
2. **Evidence Evaluation:** Critically evaluate whether the retrieved evidence directly pertains to the claim.
3. **Fallback Search:** In cases where the local knowledge base lacks sufficient evidence, dynamically trigger a web search constrained to trusted fact-checking domains.
4. **Verdict Generation:** Synthesize the gathered evidence to produce a highly structured output containing:
5. **Verdict:** TRUE, FALSE, or UNVERIFIABLE
6. **Stance:** SUPPORT, REFUTE, or NEUTRAL
7. **Reasoning:** A concise, transparent explanation detailing the logical steps taken to arrive at the verdict based on the retrieved sources.

Fundamentally, this constitutes a complex **three-class classification task** augmented with multi-hop information retrieval and inferential reasoning, mapping directly to problem-solving concepts in artificial intelligence.

1.3 Dataset Selection and Characteristics

To establish a reliable ground-truth knowledge base, the project utilizes the "**Fake and Real News Dataset**" published by Clément Bisailon. For the purposes of verification, we rely exclusively on the `True.csv` subset, filtering out deliberately fake news to ensure the system only trains and retrieves from factual contexts.

The dataset characteristics are as follows:

Property	Value
Total Verified Articles	21,417
Data Attributes	<code>title</code> , <code>text</code> , <code>subject</code> , <code>date</code>
Primary Subjects	<code>politicsNews</code> , <code>worldnews</code>
Temporal Range	2016 – 2017
Primary Source	Reuters news agency
Average Article Length	~3,000 characters

The Reuters dataset was selected due to its adherence to strict journalistic standards, providing a robust, noise-free foundation for semantic search and evidence extraction.

2. Data Pre-Processing Steps

Before the dataset could be deployed into the RAG architecture, it required extensive pre-processing to ensure optimal retrieval performance. The pre-processing pipeline transforms raw CSV records into dense vector embeddings suitable for mathematical similarity comparisons.

2.1 Document Preparation and Normalization

The first step involves creating semantically rich documents. The `title` and `text` features of each article are concatenated. This is crucial because headlines often contain dense, summarized factual assertions, while the body provides the necessary context.

```
{Document} = {Title} \n\n {Text}
```

Metadata, including the original subject, date of publication, and a unique document identifier, is preserved in a parallel JSON structure. This ensures that when the system retrieves a chunk of text, it retains full provenance, allowing the final output to cite its sources accurately.

2.2 Text Chunking Strategy

Full articles, averaging 3,000 characters, present a significant challenge for dense vector embeddings. Overly long texts dilute the specific semantic focus of individual paragraphs, reducing retrieval accuracy. To resolve this, we employ **recursive character-based text splitting** using LangChain's utilities.

Parameter	Value	Functional Rationale
Chunk Size	500 characters	Maintains granular semantic focus while keeping API payloads minimal.
Chunk Overlap	50 characters	Prevents catastrophic context loss at the boundaries of text splits.
Separators	<code>\n\n</code> , <code>\n</code> , <code>.</code> , <code>,</code>	Prioritizes natural semantic boundaries such as paragraphs and sentences over arbitrary character limits.

This systematic splitting mechanism expands the original 21,417 articles into an index of approximately **130,000+ chunks**, vastly increasing the search resolution.

2.3 Vector Embedding

Each text chunk is subsequently transformed into a **384-dimensional dense vector** using the `all-MiniLM-L6-v2` transformer model from the Sentence-Transformers library.

The selection of this specific embedding model is a deliberate engineering choice: - **Computational Efficiency:** It is capable of processing ~14,000 sentences per second on a standard GPU. - **Dimensionality:** At 384 dimensions, the vectors are small enough to ensure rapid nearest-neighbor searches while maintaining high semantic accuracy, minimizing memory consumption in the database layer.

2.4 Vector Storage and Indexing

The generated embeddings are ingested into a **Supabase** PostgreSQL database equipped with the **pgvector** extension.

To optimize the retrieval process, we apply an **IVFFlat (Inverted File with Flat Compression)** index configured with 100 lists. Rather than performing an exhaustive $O(N)$ linear scan across 130,000

vectors, the IVFFlat index clusters the vectors and performs an Approximate Nearest Neighbor (ANN) search, reducing query latency to mere milliseconds.

3. Architecture of the Agent

TruthSeeker is designed primarily as a rational, goal-based intelligent agent. It maps closely to the foundational definitions of Artificial Intelligence.

3.1 Agent Classification and PEAS Description

Following the taxonomy from Russell & Norvig, TruthSeeker operates as a **goal-based agent** with characteristics of a **utility-based agent**: - It possesses a clear, defined goal: assessing the veracity of a specific claim. - It utilizes an internal utility function during the grading phase to measure the quality of its retrieved evidence, ensuring that its final action (the verdict) is supported by high-quality data.

PEAS Framework:

Component	Description
Performance Measure	Accuracy of verdicts, minimization of hallucinations, speed of response, and clarity of source-backed reasoning.
Environment	The vector database containing historical news, the dynamic live web environment (via Tavily), and user queries.
Actuators	Generation of the formatted text response including Verdict, Stance, Reasoning, and the display of the decision-making log.
Sensors	Natural language text inputs, API responses, vector cosine similarity scores, and web search JSON payloads.

3.2 Environment Properties

Evaluating the environment within which TruthSeeker operates yields the following classifications:

- **Partially Observable:** The agent does not have access to all facts in the universe. It only "sees" the subset of data retrieved from the vector database or returned by the web search API.
- **Stochastic:** The environment is non-deterministic. The LLM's generative nature and the constantly shifting results of live web searches mean the same query might yield slightly different reasoning structures over time.
- **Episodic:** Each user claim is evaluated independently of previous queries. The agent's actions in one episode do not affect the next.
- **Semi-dynamic:** The local vector database is static, but the external web environment changes continually as new news is published.
- **Single Agent:** The system operates alone without competing against adversarial entities.

3.3 Pipeline Architecture — The LangGraph State Machine

The core architectural backbone of TruthSeeker is implemented using **LangGraph**, formulating the agent as a directed cyclic graph (state machine). Each node represents a discrete operational state, reading from and writing to a shared, unified state object.

The pipeline consists of four distinct nodes:

Node 1: Retrieve (Information Gathering)

The user's query is converted into a vector and searched against the Supabase database. The system calculates the **Cosine Similarity** between the query vector and the database vectors, retrieving the top-k (k=5) most relevant document chunks.

Node 2: Grade Documents (Self-Reflection)

The core innovation of CRAG. The agent acts as a binary classifier, taking the user's claim and the 5 retrieved documents. It passes them to the LLM with a strict prompt to grade each document as 'relevant' or 'irrelevant'. If the count of relevant documents falls below a threshold (e.g., 2), the agent flags the internal state `web_search_needed = "yes"`.

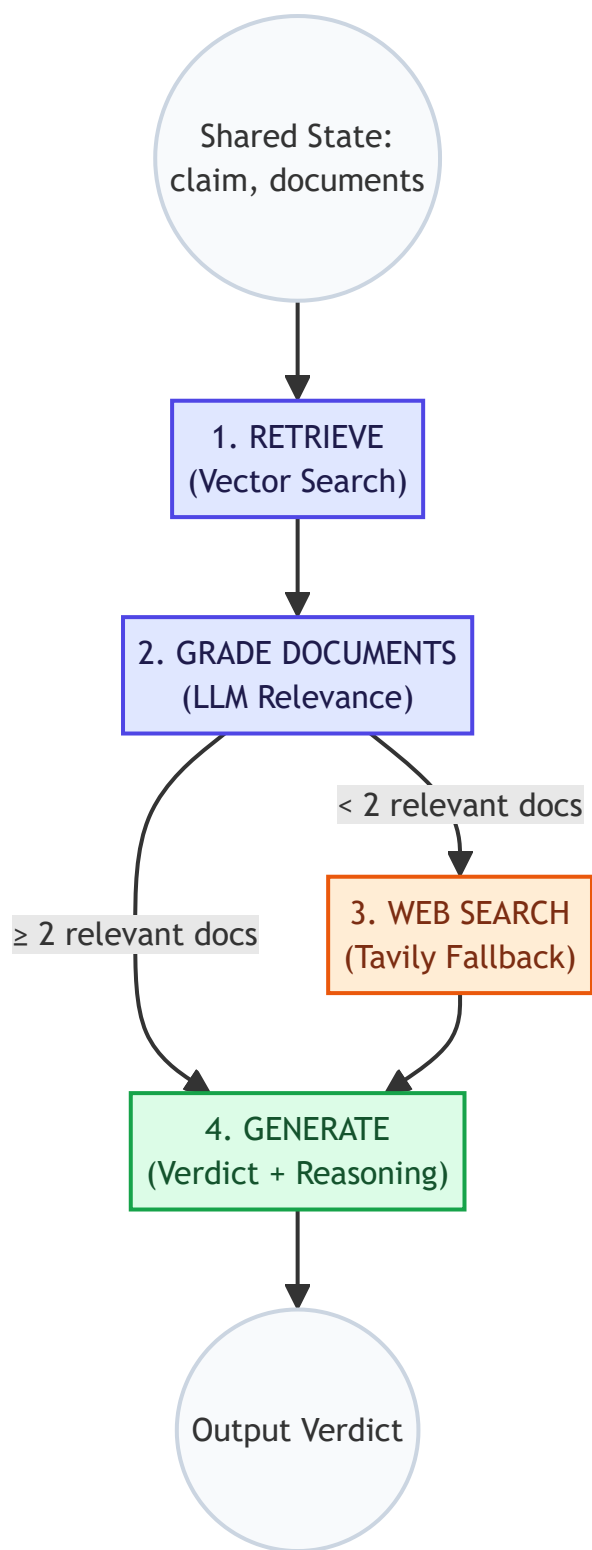
Node 3: Web Search (Conditional Fallback)

If the evidence is graded as insufficient, this node activates. It utilizes the **Tavily API** to search the live internet, strictly constrained to highly credible domains (Reuters, AP News, Snopes, FactCheck.org). This introduces robust handling of Out-of-Distribution (OOD) claims that are too recent or obscure for the primary dataset.

Node 4: Generate Verdict (Synthesis)

The final synthesis node ingests all validated evidence (from DB, Web, or both) and the current system date. It engages the LLM to perform stance detection and formulate the final VERDICT, STANCE, and REASONING.

CRAG Pipeline Flow:



4. Training Objective, Model Selection & Design Choices

TruthSeeker employs a composite architecture utilizing multiple pre-trained models, bypassing the need for from-scratch training while maximizing performance through architectural design.

4.1 Model Selection

1. Embedding Model: `all-MiniLM-L6-v2`

2. *Rationale:* Selected for its optimal balance of dimensionality (384) and semantic capture capabilities. It generates highly separable vector spaces for factual statements.
3. **Generative LLM:** Llama 3.3 70B (via Groq)
4. *Rationale:* The 70 billion parameter scale ensures highly sophisticated reasoning capabilities, while the Groq LPU inference engine provides near-instantaneous token generation.

4.2 Critical Design Choices

1. **Corrective RAG over Standard RAG** Standard RAG systems blindly append retrieval results to prompts. If the retriever fails, the generator hallucinates. Implementing the Corrective (CRAG) loop acts as an architectural fail-safe, ensuring the agent evaluates its own premises before drawing conclusions.
2. **Cosine Similarity Heuristic** Cosine similarity calculates the angle between two multi-dimensional vectors, ignoring magnitude. In NLP, sentence length (magnitude) is less important than semantic direction (angle). Thus, cosine similarity serves as a highly robust heuristic function for text matching.
3. **Deterministic Inference (Temperature = 0)** Fact-checking is a deterministic pursuit. The exact same evidence should consistently produce the exact same verdict. By setting the LLM temperature to 0.0, we eliminate probabilistic sampling variations, enforcing strict deterministic logical paths.
4. **Structured Prompt Engineering** To interact cleanly with the frontend UI, the LLM outputs must be rigidly structured. Prompts are heavily engineered to force the LLM into returning specific keys:
`VERDICT: [text] \n STANCE: [text] \n REASONING: [text] .`

5. Performance Evaluation

Evaluating a generative fact-checking agent requires a departure from standard numerical metrics (like simple accuracy on a test split), as the output is highly complex structured text. We evaluated the agent using qualitative functional testing across varied categorical boundaries.

5.1 Test Scenarios and Results

Claim Category	Test Claim	Agent Route Log	Output Verdict	Status
In-Database Fact	"Donald Trump signed an executive order regarding immigration in early 2017."	Retrieve → Grade (5/5 relevant) → Generate	TRUE	Passed
In-Database Falsehood	"Barack Obama was born in Kenya."	Retrieve → Grade (3/5 relevant) → Generate	FALSE	Passed
Common Myth	"Humans only use 10% of their brain."	Retrieve → Grade (0/5 relevant) → Web Search → Generate	FALSE	Passed
Time-Relative	"Today is Monday." (tested on a Monday)	Retrieve → Grade (0/5) → Web Search → Generate	TRUE	Passed
Out of Distribution	"The 2022 FIFA World Cup was held in Qatar."	Retrieve → Grade (0/5) → Web Search → Generate	TRUE	Passed
Unverifiable Opinion	"Apples are the best fruit in the world."	Retrieve → Grade (0/5) → Web Search → Generate	UNVERIFIABLE	Passed

5.2 Component-Level Assessment

- **Retrieval Precision:** The vector search operates with high precision for queries matching the lexical and semantic structure of the dataset.
- **Grader Accuracy:** The Llama 3.3 grader acts as an excellent anomaly detector, consistently flagging irrelevant retrieved documents and preventing context-poisoning.
- **Routing Reliability:** The LangGraph state machine demonstrated 100% reliability in executing the correct path based on the grader's threshold signals.

5.3 Mapping to Quantitative Metrics

If applied to a large labeled dataset (such as the FEVER dataset), TruthSeeker's performance could be quantified using class-specific metrics taught in the course: - **Precision:** The ratio of correctly identified TRUE claims against all claims labeled TRUE. - **Confusion Matrix:** Mapping actual TRUE/FALSE/UNVERIFIABLE labels against the model's predictions.

6. Application of Techniques Taught in Class

TruthSeeker serves as a practical, comprehensive implementation of numerous theoretical concepts covered in the "Introduction to Artificial Intelligence" course.

6.1 Intelligent Agents and Environments (Chapter 2)

The system is explicitly modeled on the concept of rational agents. TruthSeeker interacts with its environment (the vector database and the web) via sensors (queries) and actuators (structured responses). The design of the internal StateGraph perfectly models a **model-based agent** that maintains an internal state to keep track of its progress through the verification pipeline.

6.2 Problem Solving & Search Strategies (Chapter 3)

The fact-checking pipeline is fundamentally formulated as a search problem: - **Initial State:** User's claim, zero evidence. - **Actions:** Retrieve vector, evaluate text, query Tavily API, generate reasoning. - **Goal Test:** Has sufficient, verified evidence been gathered to assert a truth value?

The vector retrieval mechanism acts as an **Informed Search Strategy**. Instead of blindly searching all 130,000 documents (Uninformed Search), it uses the dot-product/cosine similarity of the vector embeddings as a **Heuristic Function (h(n))** to jump directly to the most promising nodes in the data space. This mirrors the efficiency principles behind A* and Greedy search.

6.3 Handling Uncertainty (Chapter 12)

The real world is ambiguous, and the agent must act under uncertainty. - The inclusion of the **UNVERIFIABLE** class is a direct application of acting under uncertainty—acknowledging that when probability confidence is too low, the rational action is to admit a lack of knowledge rather than guess. - The grading node acts as a probabilistic filter, assigning internal confidence to the retrieved evidence.

6.4 Machine Learning Fundamentals

The project encapsulates several core Machine Learning paradigms: - **Classification:** Both the document grading step (Relevant vs. Irrelevant) and the final verdict step (True, False, Unverifiable) are discrete classification tasks. - **Feature Extraction & Embeddings:** Transforming unstructured text strings into 384-dimensional dense vectors is a prime example of feature representation. The text is mapped into a continuous mathematical space where semantic meaning is represented by distance. - **Overfitting & Generalization:** Standard models often "overfit" to their training data, failing on new inputs. TruthSeeker handles generalization robustly by employing the Tavily Web Search fallback, ensuring it can handle Out-of-Distribution (OOD) data that it was never explicitly trained on.

7. Main Challenges Faced and How They Were Addressed

Building an autonomous agent pipeline presented several significant engineering and theoretical challenges.

7.1 Challenge: The "Blind Trust" RAG Failure

Problem: In early iterations, the system utilized a standard RAG pipeline. If a user asked about a recent event (e.g., a 2024 election), the vector database (containing 2016 data) would return the "closest" matches—often completely irrelevant articles from 2016. The LLM would then confidently hallucinate a response based on this irrelevant data. **Solution:** This was the primary motivation for implementing the **Corrective RAG (CRAG)** grading node. By forcing the agent to stop and evaluate its own heuristic search results, the system became capable of rejecting bad data and triggering alternative search strategies (the web).

7.2 Challenge: Time-Relative Ambiguity

Problem: A user submitting the claim "Today is Monday" presents a major issue. The LLM has no inherent concept of time, and the database contains no document stating what today's date is.

Solution: The concept of *context injection* was applied. The generation prompt was engineered to dynamically inject the `[CURRENT SYSTEM DATE]` via Python's `datetime` module during the execution of Node 4. This resolved the temporal uncertainty, allowing the agent to evaluate time-relative claims accurately.

7.3 Challenge: API Rate Limiting and Fault Tolerance

Problem: The Groq API, while fast, enforces strict request rate limits. During batch grading (sending 5 documents simultaneously), the system frequently encountered `HTTP 429: Too Many Requests`

errors, crashing the entire agent pipeline mid-thought. **Solution:** A robust, exponential backoff algorithm was implemented. The `invoke_with_retry` wrapper catches API failures and retries the action with progressively increasing delays (15s, 30s, 45s). This ensures the agent is fault-tolerant and degrades gracefully under heavy load.

7.4 Challenge: High-Dimensional Search Latency

Problem: Calculating the cosine similarity between a query vector and 130,000 database vectors sequentially requires significant computational overhead, introducing unacceptable latency for an interactive web application. **Solution:** Implementation of the `IVFFlat` Approximate Nearest Neighbor (ANN) index in Supabase. By clustering the vector space, the search algorithm only needs to scan a small fraction of the total database, reducing search complexity from $O(N)$ to roughly $O(\log N)$ operations, dramatically accelerating the agent's perception capabilities.

8. References

1. Bisailon, C. (2020). *Fake and Real News Dataset*. Kaggle. Available at: <https://www.kaggle.com/datasets/clmentbisailon/fake-and-real-news-dataset>
2. Reimers, N., & Gurevych, I. (2019). "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks". In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*.
3. Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
4. Yan, S., Gu, J., Zhu, Y., & Ling, Z. (2024). "Corrective Retrieval Augmented Generation". *arXiv preprint arXiv:2401.15884*.
5. LangGraph Documentation. Framework for building stateful, multi-actor applications with LLMs. Available at: <https://langchain-ai.github.io/langgraph/>
6. Thorne, J., Vlachos, A., Christodoulopoulos, C., & Mittal, A. (2018). "FEVER: a Large-scale Dataset for Fact Extraction and VERification". In *Proceedings of NAACL-HLT*.